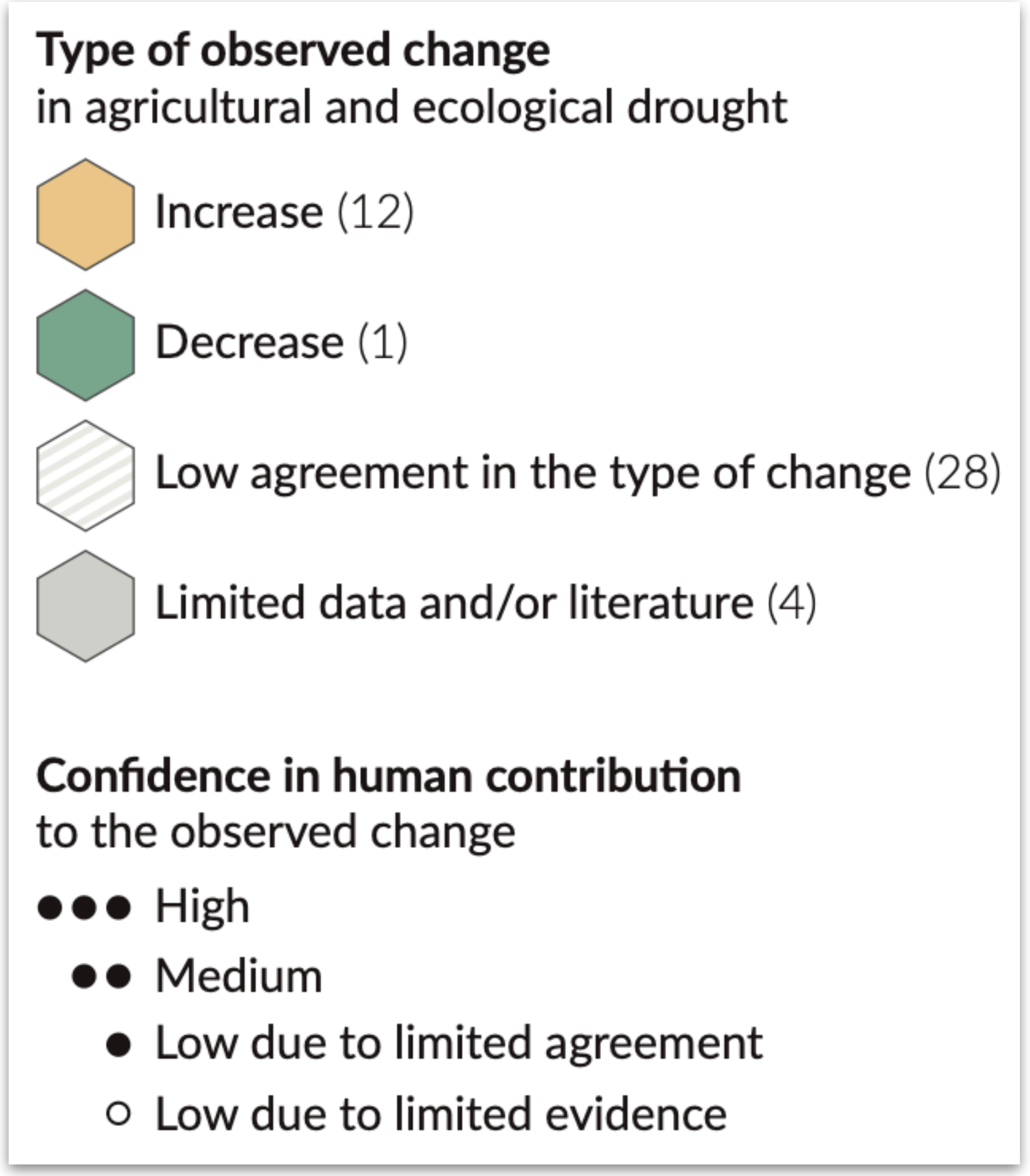
Fluid: towards transparent, self-explanatory research outputs

Joe Bond[*], **Cristina David**[*], **Minh Nguyen**[*], **Roly Perera**[*†]

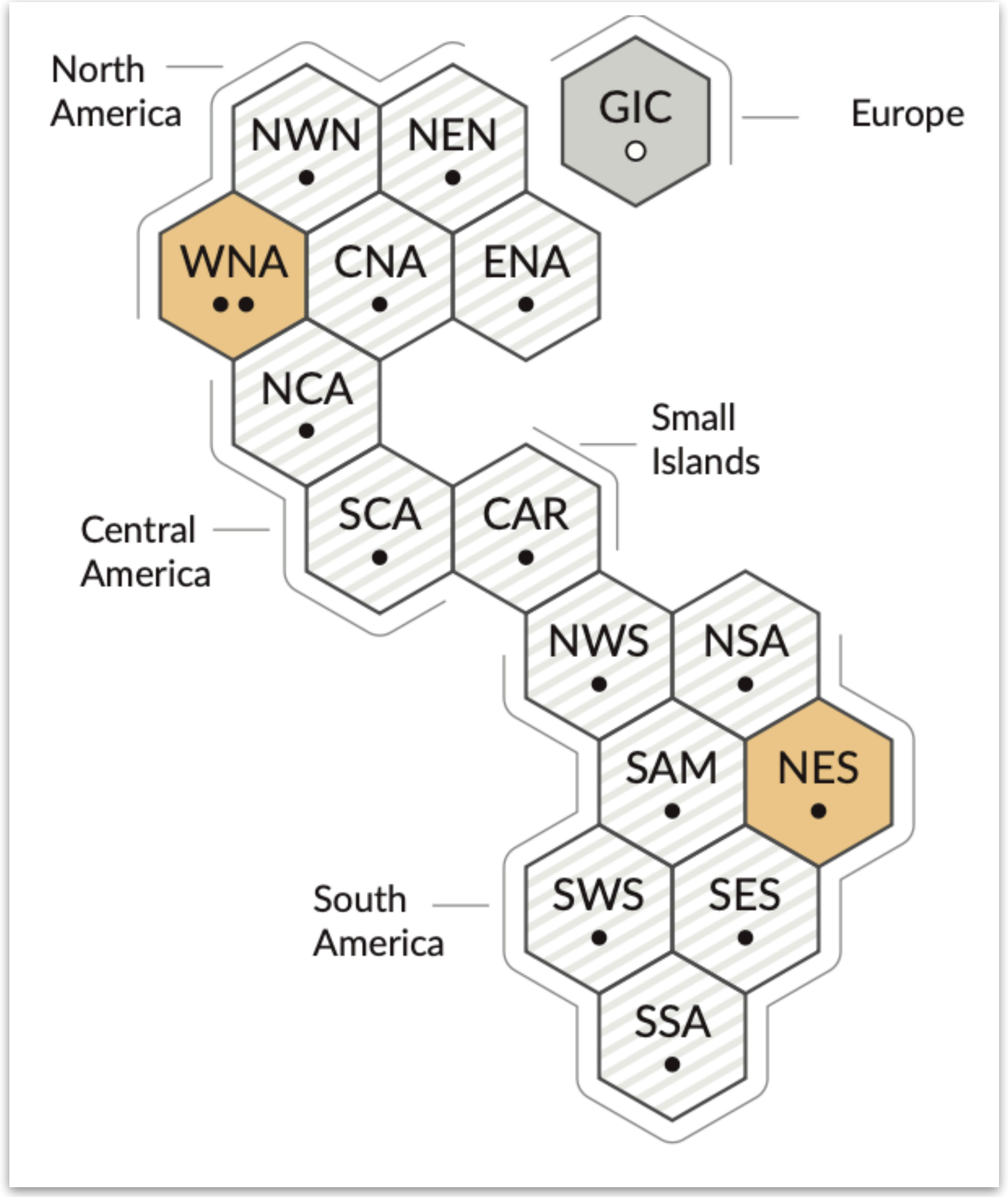
[*]University of Bristol [†]Institute of Computing for Climate Science, University of Cambridge

Understanding, trusting and evaluating visual information

Matters to policy makers, scientists, journalists, educators (and just regular people)



Legends and captions of limited value



Informative in a certain sense but
- what are we looking at?
- where did it come from?
- why should we should trust it?

Familiar problem for traditional print media, but what about digital media?

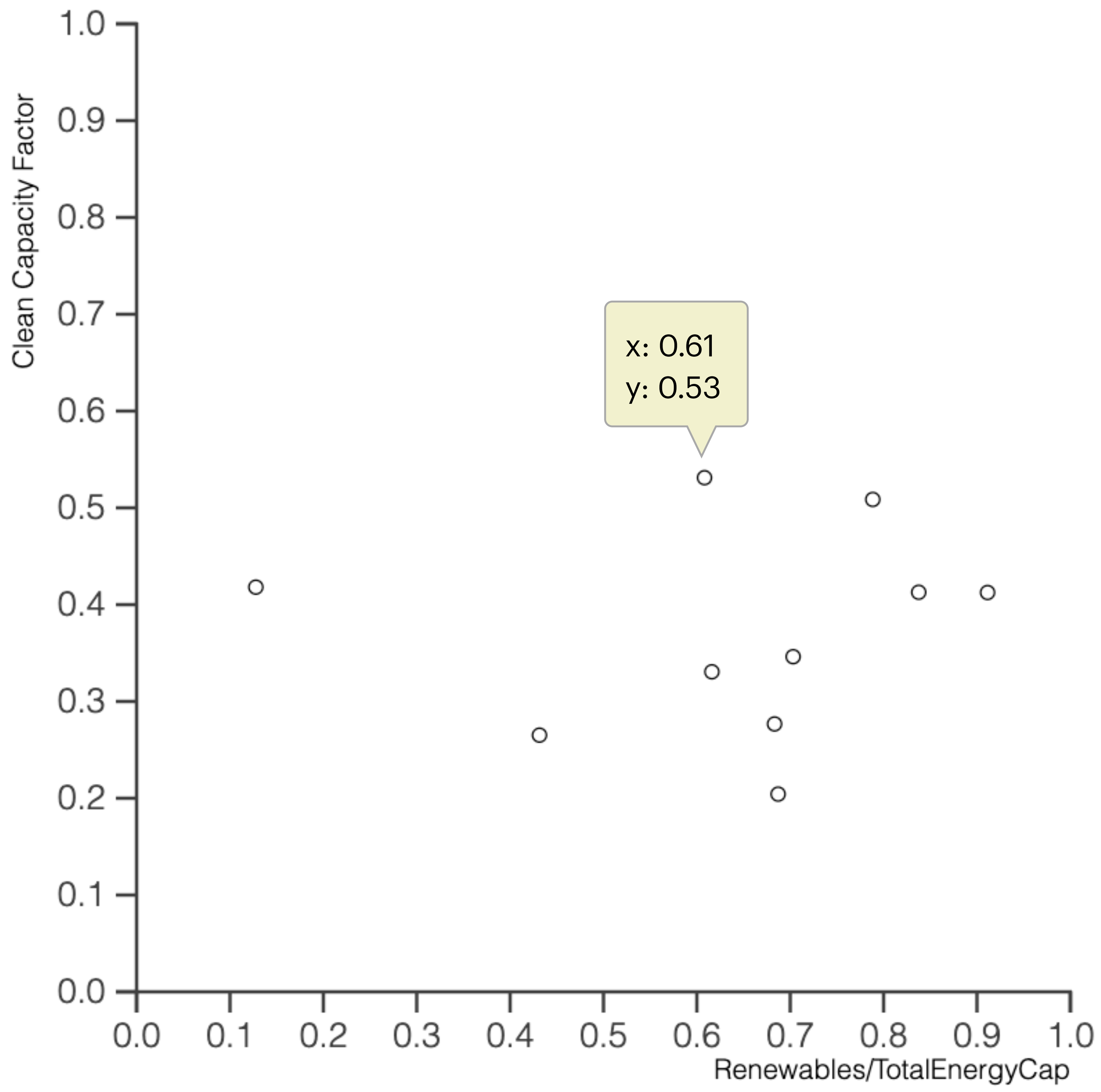
Can PL help?

> Heating of the climate system has caused global mean sea level rise through ice loss on land and thermal expansion from ocean warming. Thermal expansion explained 50% of sea level rise during 1971–2018, while ice loss from glaciers contributed 22%, ice sheets 20% and changes in land-water storage 8%. The rate of ice-sheet loss increased by a factor of four between 1992–1999 and 2010–2019. Together, ice-sheet and glacier mass loss were the dominant contributors to

Supporting text references computed data

IPCC, 2021: Summary for Policymakers

Towards transparent, self-explanatory data visualisation



Clean energy efficiency against proportion of renewable energy cap

Questions we might ask

What year is this data from?
Is each circle a country?
What do *x* and *y* represent?
What energy types count as "renewable"?

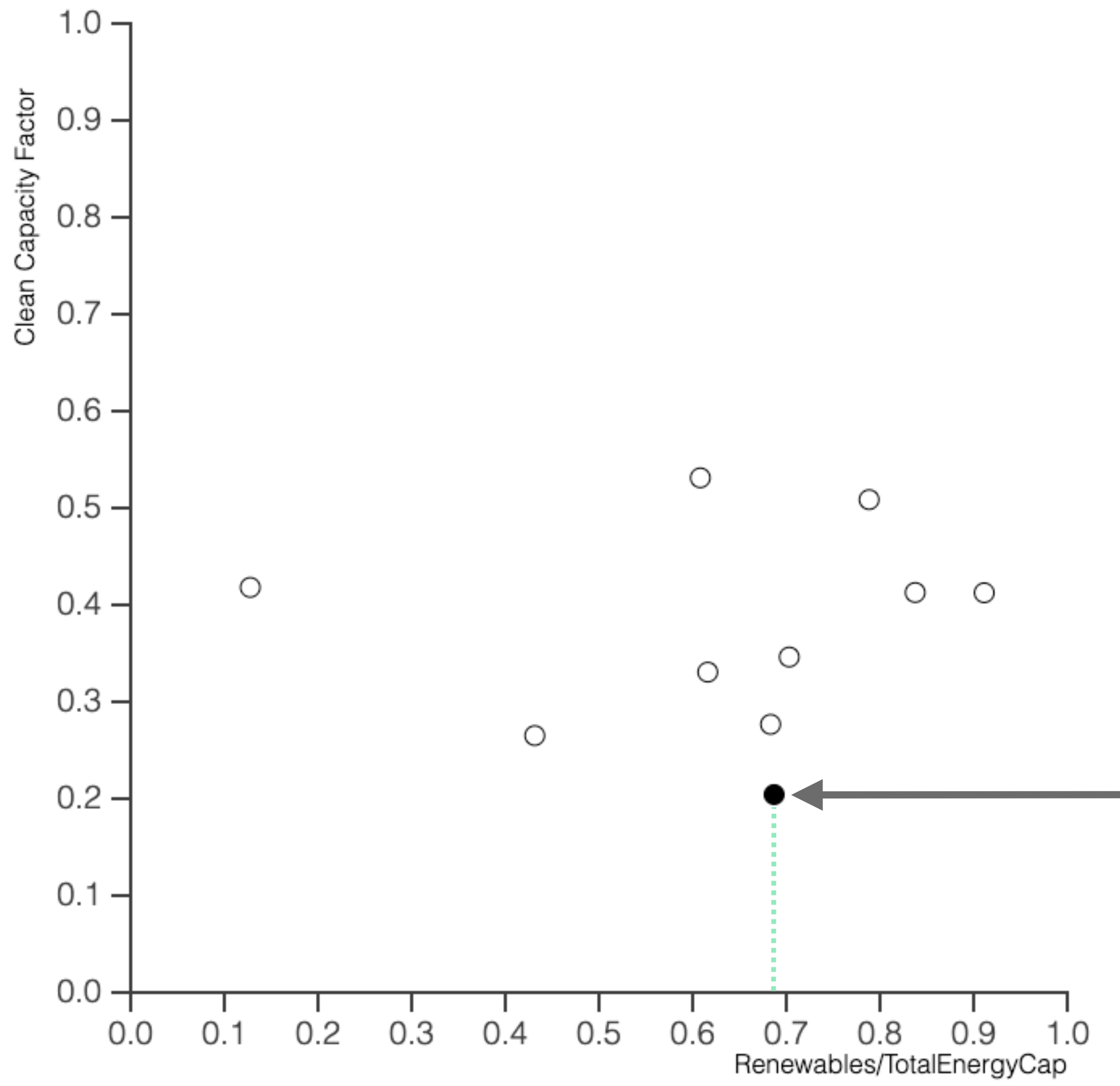
Viz libraries provide tooltips which clarify the outputs, but what about the relationship to inputs?

Data and source code help but reverse-engineering relationships is not easy

Demo: related inputs

Demo: related inputs

Related inputs operator ▽△
is not idempotent



Clean energy efficiency against proportion of renewable energy cap

Mediating output answers that question and induces a demand on other data — here only the x-coordinate is selected

Primary selection asks what demands this data?

non_renewables

	coalCap	coalGen	country	gasCap	gasGen	nuclearCap	nuclearGen	petrolCap	petrolGen	year
57	405.5	345.13	JPN	747.84	425.37	333.23	49.11	241.69	53.78	2018

renewables

	capacity	country	energyType	output	year
225	18.22	JPN	Bio	30.13	2018
226	246.51	JPN	Hydro	81.11	2018
227	491.96	JPN	Solar	62.11	2018
228	30.66	JPN	Wind	6.44	2018

non_renewables

	coalCap	coalGen	country	gasCap	gasGen	nuclearCap	nuclearGen	petrolCap	petrolGen	year
57	405.5	345.13	JPN	747.84	425.37	333.23	49.11	241.69	53.78	2018

renewables

	capacity	country	energyType	output	year
225	18.22	JPN	Bio	30.13	2018
226	246.51	JPN	Hydro	81.11	2018
227	491.96	JPN	Solar	62.11	2018
228	30.66	JPN	Wind	6.44	2018

Adding capacity for any renewable makes nuclearGen and all renewable output relevant too

Fluid: a transparent programming language

```
MultiPlot {|
   "bar-chart" :=
      let totalFor c rows =
         sum [ row.output | row ← rows, row.country == c ];
      let data2015 = [ row | row ← renewables, row.year == 2015 ];
         countryData = [ { x: c, y: totalFor c data2015 }
                       | c ← ["China", "USA", "Germany"] ]
      in BarChart {
         caption: "Total output by country",
         data: countryData
      },
   "line-chart" :=
      let series type country = [
         { x: row.year, y: row.output }
         | year ← [2013..2018], row ← renewables,
         row.year == year, row.energyType == type, row.country == country
      ] in LineChart {
         caption: "Output of USA relative to China",
         plots: [
            LinePlot { name: type, data: plot }
            | type ← ["Bio", "Hydro", "Solar", "Wind"],
            let plot = zipWith (fun p1 p2 → { x: p1.x, y: p1.y / p2.y })
                               (series type "USA") (series type "China")
         ]
      }
|}
```

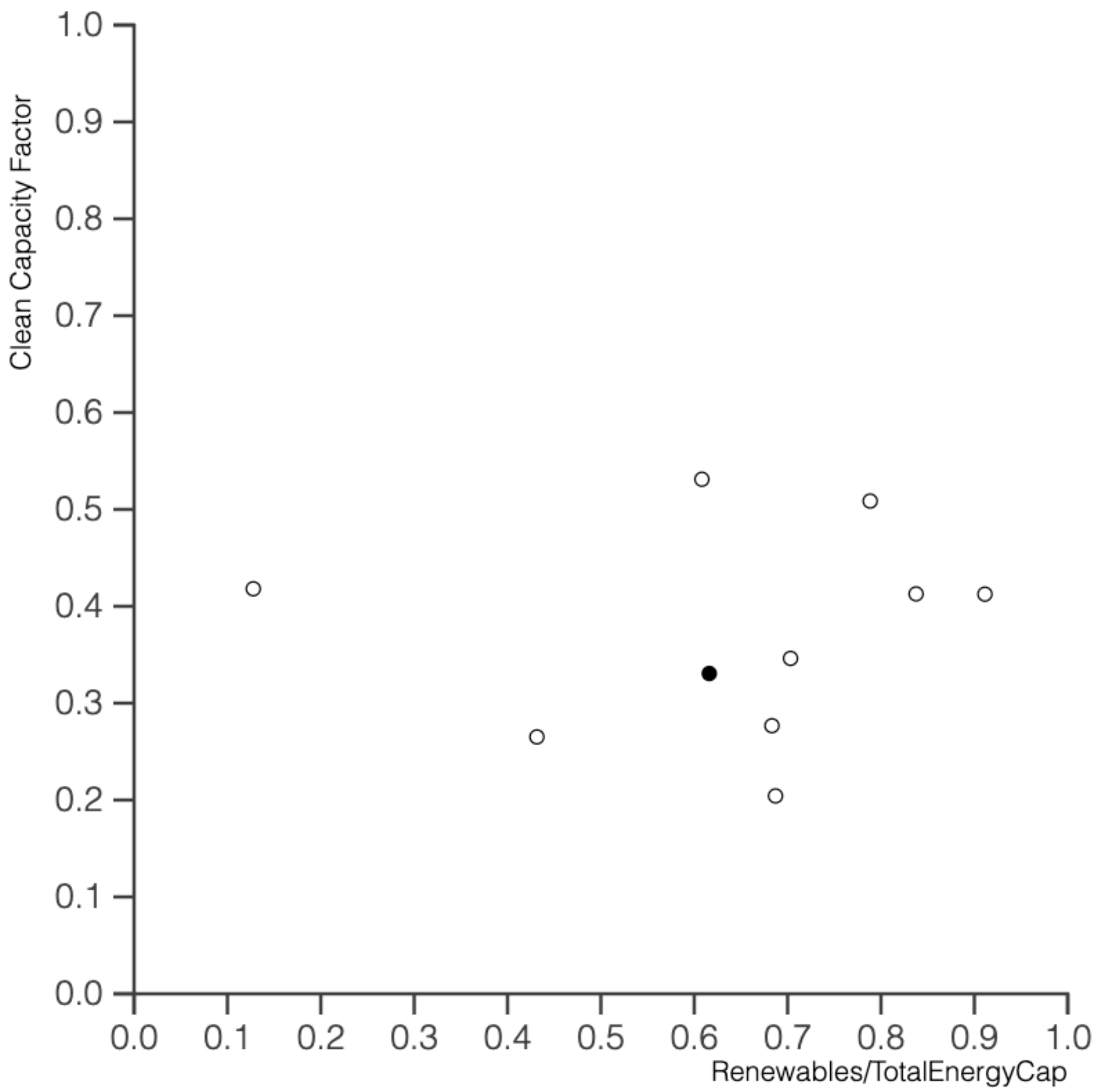
Untyped, interpreted functional language
Records, dictionaries, data types, matrices
No effects to speak of (only errors)

Program evaluates to result plus dynamic dependence graph G

Use reachability in G and G^{-1} to implement demands and demanded by operators

d3.js front end to visualise inputs, outputs with selection information

Related inputs



Clean energy efficiency against proportion of renewable energy cap

renewables

	capacity	country	energyType	output	year
205	75.86	CHN	Bio	93.73	2018
206	2823.09	CHN	Hydro	1198.89	2018
207	1535.28	CHN	Solar	176.9	2018
208	1617.62	CHN	Wind	365.8	2018

non_renewables

	coalCap	coalGen	country	gasCap	gasGen	nuclearCap	nuclearGen	petrolCap	petrolGen	year
52	8555.54	4763.92	CHN	710.7	215.5	391.22	295	55.19	56.4	2018

Dual to brushing and linking feature from data visualisation

Not full intensional explanations (future work), but “data transparent”

Conjugate operators for related inputs

What about correctness and minimality of $\triangleright$ and $\triangleleft$?

Diagram illustrating *conjugacy* (maybe self-conjugacy)

Definition (Tarski)

Diagram(s) showing $\triangleright$ composed with $\langle \triangleleft, \mathrm{id} \rangle$ to return “mediating data” (conjugate pairs admit products, maybe give construction)

Related outputs, revisited

Explored dual idea in POPL 2022

Swap one operator by its conjugate to compute related outputs:

$$C \xrightarrow{\nabla_f} A \xleftarrow{\nabla_g} B$$

$$C \xleftarrow{\Delta_f} A \xleftarrow{\nabla_g} B$$

But doesn't lend itself to related inputs:

$$A \xleftarrow{\nabla_f} C \; ↯ \; B \xleftarrow{\Delta_g} A$$

More general form is composition of product view with its own conjugate

$$B \times C \xleftarrow{\Delta} A \xleftarrow{\nabla} B \times C$$

with (optional) projections to disregard irrelevant views

$$C \xleftarrow{\Delta_{snd}} B \times C \xleftarrow{\Delta} A \xleftarrow{\nabla} B \times C \xleftarrow{\nabla_{fst}} B$$

Projection clamps demand of discarded part at $\perp$

$$\nabla_{fst}(b) = (b, \perp) \qquad \Delta_{fst}(b,c) = b$$

Recovers related outputs but amenable to related inputs

$$A \xleftarrow{\nabla} B \times C \xleftarrow{\Delta} A$$

Focused queries via projection operators

Unfocused query

output

5	4	2	5	2
3	1	2	-1	-2
3	0	1	0	-1
2	1	-2	0	0
1	0	-1	-1	-2

filter

-2	-1	0
-1	1	1
0	1	2

input_image

15	13	6	9	16
12	5	15	4	13
14	9	20	8	1
4	10	3	7	19
3	11	15	2	9

Related outputs with respect to both inputs

Focused query

output

5	4	2	5	2
3	1	2	-1	-2
3	0	1	0	-1
2	1	-2	0	0
1	0	-1	-1	-2

input_image

15	13	6	9	16
12	5	15	4	13
14	9	20	8	1
4	10	3	7	19
3	11	15	2	9

Related outputs with respect to input_image only

Composing $\triangleleft$ with projection operator discards irrelevant inputs

Availability of discarded inputs clamped at $\top$

Focused queries via projection operators

Projection operators turn out to be super-useful

Related inputs

compose $\rhd$ with projection to disregard irrelevant outputs

(clamps their demand at $\bot$)

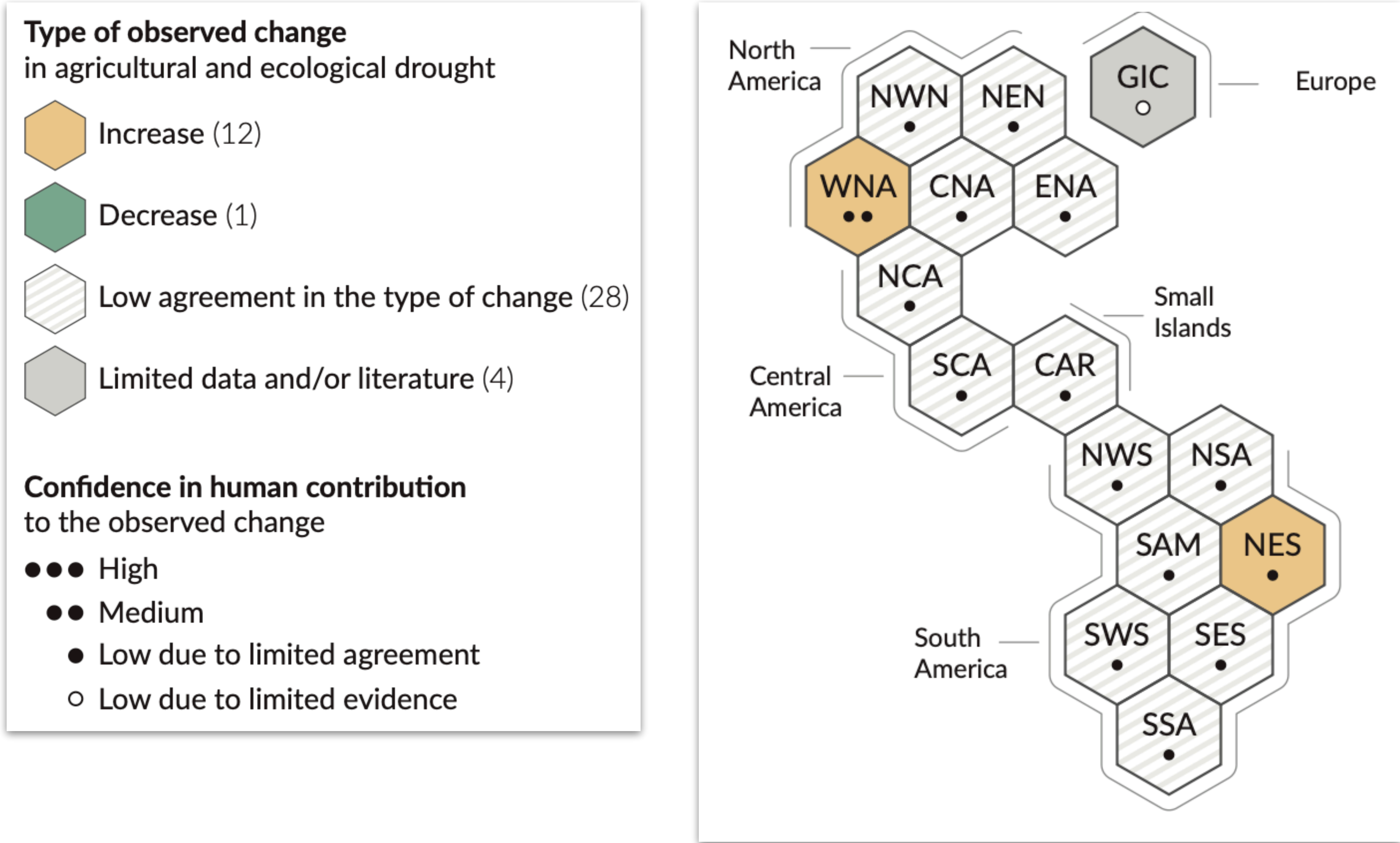
"which inputs are related with respect to these particular outputs" (e.g. x vs y coordinate)

compose $\lhd$ with projection to disregard irrelevant sources

(clamps their availability at $\top$)

"what of these particular inputs are related?"

Use case



Focus on layer of visualisation, e.g. confidence in human contribution, to fix query context

What next

Language

Type system?
Effect system?
Units of measure
Multidimensional arrays
Markdown + quasiquotation

Applications

Online articles or long-form essays presenting climate science with transparent and self-explanatory charts and figures

http://f.luid.org
https://github.com/explorable-viz/fluid